
Work Report:

Precision Medicine: Finding exRNA/cfRNA biomarker panel for cancer diagnosis

For Coursework of *Bioinformatics*

Huang Xiran (黄悉然), School of Life Sciences, Tsinghua University, China

Stu. ID: 2025010957; email: huangxr25@mails.tsinghua.edu.cn

1. Introduction

Extracellular RNAs (exRNAs) have emerged as a promising class of biomarkers for non-invasive disease detection. Although RNA molecules are generally susceptible to degradation by ubiquitous ribonucleases, various forms of RNA—including microRNAs, messenger RNAs, long non-coding RNAs, and circular RNAs—have been consistently detected in biofluids such as serum, saliva, and urine. Their stability is largely attributed to encapsulation in microvesicles or exosomes, tight association with RNA-binding proteins, or intrinsic structural features such as circular conformation that confer resistance to exonucleolytic decay. Importantly, the composition and modification state of exRNAs often reflect the physiological and pathological status of their cells of origin. Because these biofluids can be obtained through minimally invasive procedures, exRNAs present a compelling resource for

the development of liquid biopsy assays, with applications ranging from prenatal testing to early cancer detection and monitoring of disease progression.

In this study, I analyzed two provided plasma-derived RNA-seq datasets generated from long RNA and small RNA libraries. These datasets included samples from patients with multiple cancer types—colorectal, gastric, lung, oesophageal, liver, prostate, and pancreatic—as well as healthy donors. Both datasets had been pre-processed through standard quality control and alignment pipelines prior to my analysis. My objective was to construct a robust machine-learning workflow to evaluate the discriminative power of exRNA expression profiles in distinguishing cancer patients from healthy individuals and to identify a compact, biologically interpretable feature panel.

2. Data Download

The raw sequencing data were originally stored on the Tsinghua Cloud platform and were not pre-staged on the cluster. Given the large total size of the datasets, direct

transfer via conventional tools was impractical. To facilitate access for myself and other course participants, I implemented a custom Python download script that interfaces with the Tsinghua Cloud API to recursively

retrieve the entire shared directory structure while preserving the original folder hierarchy. The script uses the `requests` library to authenticate and fetch files via the public share link, supports resumption of interrupted downloads by skipping existing files, and was designed to run as a background process on the cluster.

This approach allowed the transfer to proceed reliably in the background while logging all activities for monitoring and troubleshooting. Once completed, the data were made available under the shared cluster directory for subsequent analysis by other group members.

The download was executed using the following command:

```
nohup python -u thu_cloud_download.py \  
  "https://cloud.tsinghua.edu.cn/d/e3215eb0090f4b3dac1e/?p=%2FFiles%2FPART_V%2FAssignment-1%20exSEEK&mode=list" \  
  /home/data/public/Assignment-1 \  
  > download.log 2>&1 &
```

3. Matrix Processing

Prior to downstream machine learning, the raw count matrices from both long RNA and small RNA datasets underwent a standardised preprocessing pipeline, including low-expression filtering, normalization, batch

effect assessment and correction where necessary, and scaling. The same procedural framework was applied to both data types, with the small RNA dataset requiring additional batch correction steps due to detectable technical variation.

3.1 Matrix Processing of Long RNAs

The long RNA count matrix contained 373 samples and was filtered to retain genes with sufficient expression across samples using `filterByExpr` from the `edgeR` package. Normalization was performed with the TMM (trimmed mean of M-values) method via `calcNormFactors`, followed by log-counts-per-million (logCPM) transformation using `cpm` with `log = TRUE`. A relative log expression (RLE) plot was generated to inspect the overall distribution of the normalized data.

To assess potential batch effects, principal component analysis (PCA) was performed on the transposed

logCPM matrix. Samples were colored by library preparation type (library column in metadata). The PCA plot showed no obvious clustering by this technical factor, indicating that batch effects were not a major concern for the long RNA dataset. Consequently, no batch correction was applied.

Finally, the expression matrix was transposed to a samples-as-rows format and standardized using `scale` with `center = TRUE` and `scale = TRUE`, yielding a scaled matrix suitable for machine learning algorithms that require feature normalization.

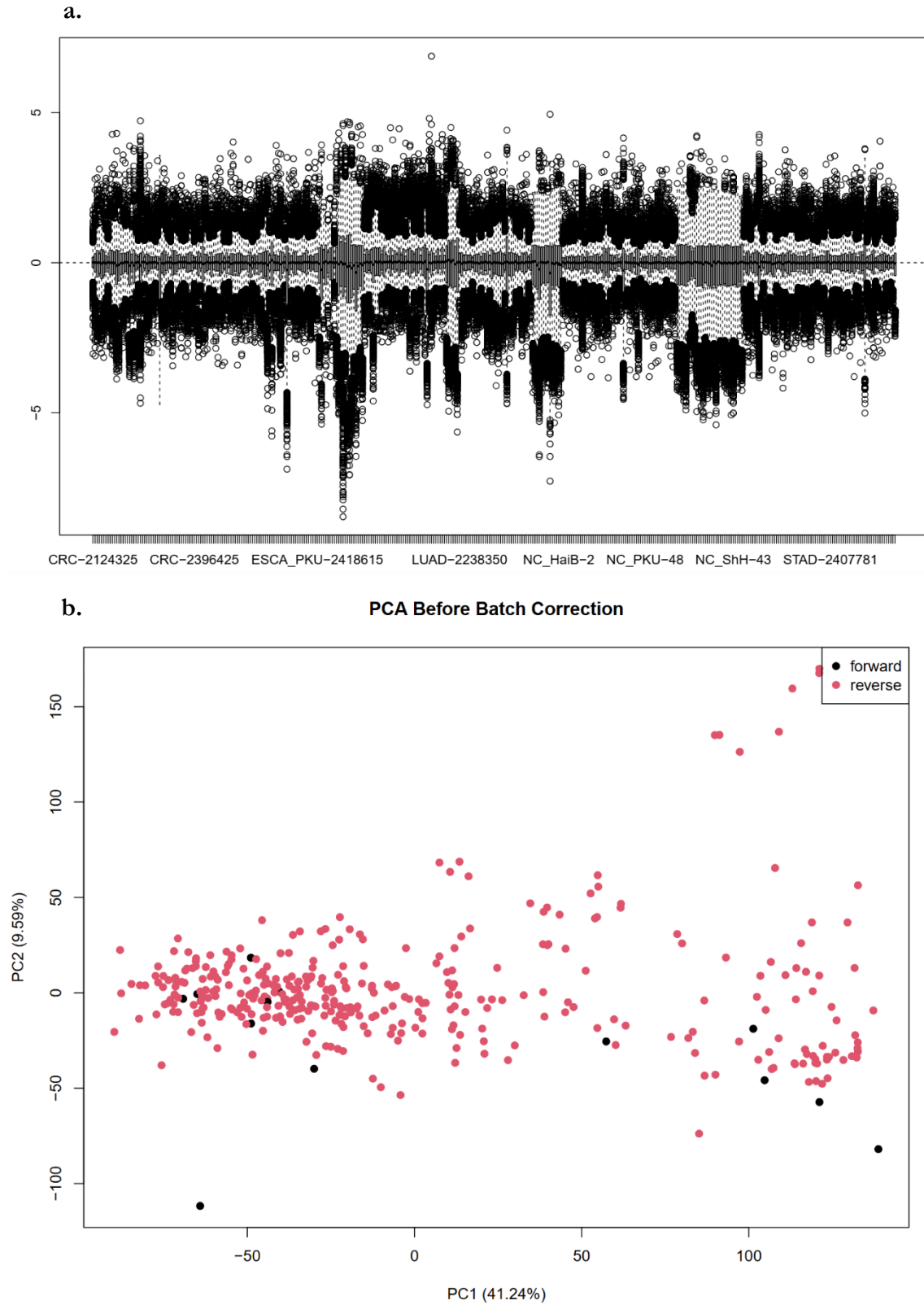


Figure 1. Quality assessment of the long RNA dataset. (a) Relative log expression (RLE) plot of the TMM-normalized long RNA expression data. The distribution of log-fold changes relative to the median sample is shown across all samples. The boxes are centered around zero for most samples, indicating effective normalization. (b) Principal component analysis (PCA) of the long RNA expression matrix. Samples are colored by library preparation type (reverse vs. forward). No clear separation between the two groups is observed along the first two principal components, suggesting that technical variation from library preparation does not dominate the overall expression structure. Accordingly, no batch correction was applied to the long RNA dataset.

3.2 Matrix Processing of Small RNAs

The small RNA count matrix comprised 192 samples. The same filtering and TMM normalization steps were applied as described for the long RNA dataset, resulting in a logCPM matrix. An RLE plot was generated for quality assessment.

Batch effect assessment was more critical for this dataset, as the metadata contained multiple technical covariates: `RNA Isolation batch`, `library preparation day`, and `gel cut size selection`. PCA was performed and samples were colored by each of these three factors separately. The resulting plots revealed noticeable clustering by both `RNA Isolation batch` and `library preparation day`, suggesting the presence of substantial technical variation that could confound biological signals.

To address this, I applied RUVg (remove unwanted variation using control genes) from the RUVSeq

package. Since no predefined housekeeping genes were available for small RNA data, I adopted an empirical control strategy by selecting the 20% of features with the smallest coefficient of variation across all samples. These features were used as negative controls to estimate and remove unwanted variation, with `k = 2` latent factors. After RUVg correction, PCA plots were re-examined, and the previously observed batch-associated clustering was substantially reduced. The improvement was further confirmed by boxplots of sample-wise sum of logCPM, which showed that the between-batch differences in total expression levels were markedly compressed.

Following batch correction, the expression matrix was transposed and standardized using the same scaling procedure as for the long RNA dataset.

4. Machine Learning

After preprocessing, I applied a supervised machine learning pipeline to each dataset to evaluate the discriminative power of the expression features and to identify a compact feature panel for cancer classification. The pipeline consisted of feature selection, model training with hyperparameter tuning, and performance evaluation on held-out test data.

For both datasets, I used a filter-based feature selection method: for each feature, I performed a two-sided Welch's t-test comparing expression levels between cancer and healthy samples in the training set, and ranked features by their p-values. This approach is

computationally efficient and independent of the subsequent classifier, providing an unbiased initial ranking of features.

The classifier used throughout was XGBoost, a gradient-boosted tree algorithm that handles non-linear relationships and interactions among features effectively. Hyperparameters were optimized via five-fold cross-validation on the training set, using AUC as the evaluation metric. The final model was evaluated on the test set, with performance reported as AUC, accuracy, sensitivity, and specificity.

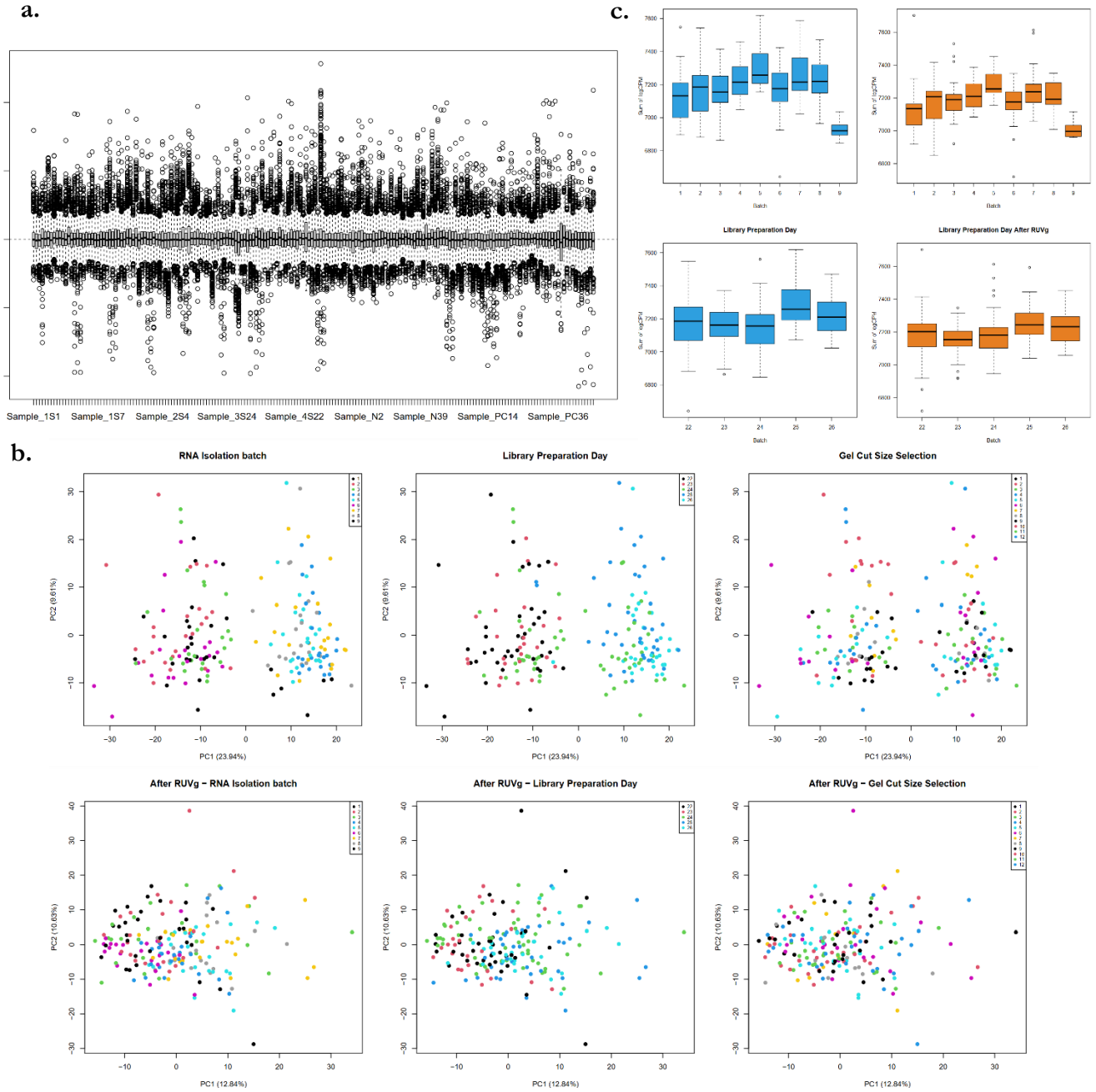


Figure 2. Quality assessment and batch effect removal for the small RNA dataset. (a) Relative log expression (RLE) plot of the TMM-normalized small RNA expression data. The distribution of log-fold changes relative to the median sample is shown across all samples. The boxes are centered around zero for most samples, indicating effective normalization. (b) Principal component analysis (PCA) of the small RNA expression matrix before (upper panels) and after (lower panels) RUVg correction. Samples are colored by three technical covariates: RNA Isolation batch (left), library preparation day (middle), and gel cut size selection (right). Prior to correction, clear clustering by RNA Isolation batch and library preparation day is observed along the first two principal components, suggesting the presence of substantial batch effects. After RUVg correction with $k = 2$ latent factors and empirical control genes (20% of features with the smallest coefficient of variation), the batch-associated clustering is substantially reduced across all three covariates, indicating successful removal of unwanted technical variation. (c) Boxplots of sample-wise sum of logCPM before (top) and after (bottom) RUVg correction, grouped by RNA Isolation batch (left) and library preparation day (right). Before correction, marked differences in median total expression levels are observed between batches. After correction, the between-batch differences are visibly compressed and the overall distribution of total expression levels becomes more homogeneous, further confirming the effectiveness of the RUVg procedure.

4.1 Machine Learning of Long RNAs

The long RNA training set consisted of 262 samples and approximately 10,000 features after preprocessing. Feature ranking by t-test yielded a list of genes ordered by differential expression significance. To determine the optimal number of features to retain, I evaluated the cross-validated AUC across a range of feature counts: 1–10, 20, 30, 50, 100, 200, 500 and 1000. The resulting curve showed that AUC increased rapidly and plateaued around 200 features, which was selected as the optimal panel size.

The XGBoost model was trained using the top 200 features. Cross-validation selected 104 boosting rounds before early stopping. On the test set, the model achieved an AUC of 0.891, with an accuracy of 85.6%, sensitivity of 58.6% for the healthy class, and specificity of 95.1% for the cancer class. The ROC

curve is presented in Figure 3b.

To visualize the expression patterns of the selected features, I generated a heatmap of the top 200 genes on the test set samples, grouped by class. The heatmap shows a clear separation between healthy and cancer samples, with distinct expression clusters across the two groups.

Functional enrichment analysis was performed using gprofiler2 on the top 200 genes, querying GO biological processes, GO cellular components, GO molecular functions, KEGG pathways, and Reactome pathways. The top 20 significantly enriched terms are shown as a bubble plot. Notably, the most enriched terms were associated with **translation**, **cytoplasmic translation**, and **ribosome biogenesis**, consistent with the known hallmarks of cancer cell proliferation.

4.2 Machine Learning of Small RNAs

The small RNA training set comprised 135 samples and 1,953 features (miRNAs and piRNAs). The same feature ranking procedure was applied. To identify the optimal feature count, I evaluated cross-validated AUC across feature counts of 1–10, 20, 30, 50, 100, 150, 200, 300 and 500. The AUC curve peaked at 20 features, which were selected for final model training.

The XGBoost model was trained on the top 20 features, with cross-validation selecting 39 boosting rounds. On the test set, the model achieved an AUC of 0.686, with an accuracy of 80.7%, sensitivity of 53.3% for the healthy class, and specificity of 90.5% for the cancer class. The ROC curve is shown in Figure 4b.

A heatmap of the top 20 features on the test set samples showed moderate separation between the two classes, though the clustering was less pronounced than in the long RNA dataset.

Functional enrichment analysis was performed on the 7 miRNAs using [RNAenrich](#), querying KEGG and Reactome pathways. The top significantly enriched pathways included **signaling pathways regulating pluripotency of stem cells**, **Hippo signaling pathway**, **endometrial cancer**, and **hepatocellular carcinoma**. These results indicate that the identified miRNAs are functionally associated with cancer-related signaling and developmental pathways, consistent with their known roles in tumorigenesis and metastasis.

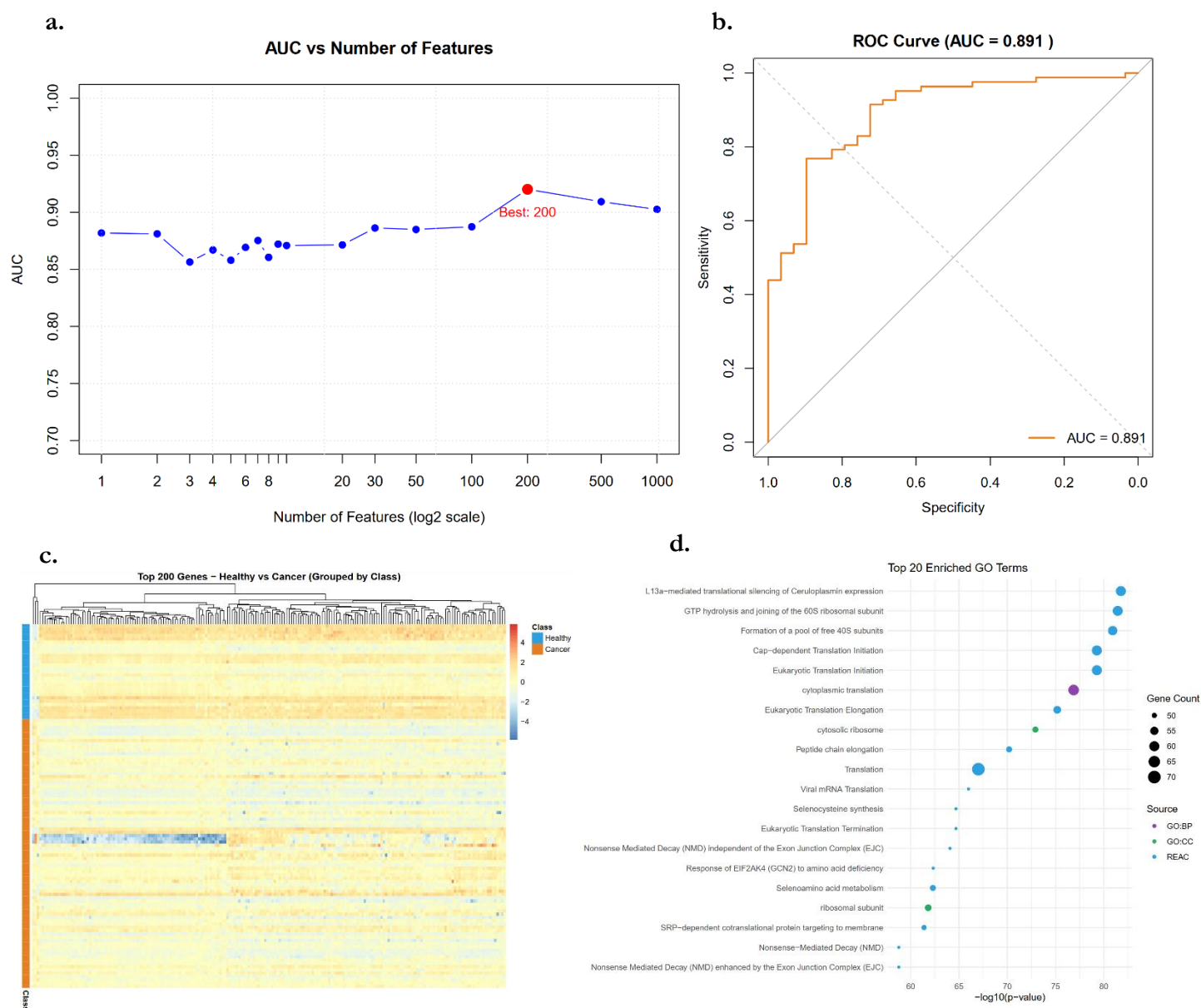


Figure 3. Machine learning results for the long RNA dataset. (a) Cross-validated AUC as a function of the number of top-ranked features included in the XGBoost model. The curve rises rapidly and plateaus around 200 features, which were selected as the optimal panel size. (b) ROC curve of the final XGBoost model evaluated on the held-out test set. The model achieved an AUC of 0.891, indicating strong discriminative power between cancer and healthy samples. (c) Heatmap of the top 200 selected genes on the test set samples, grouped by class (Healthy vs. Cancer). Samples are shown in rows and genes in columns, with expression values scaled by column. Clear separation between the two classes is observed, with distinct expression clusters corresponding to each group. (d) Bubble plot of the top 20 significantly enriched terms from GO biological processes, GO cellular components, GO molecular functions, KEGG pathways, and Reactome pathways for the top 200 genes. The size of each bubble represents the number of genes annotated to the term, and the color indicates the source database. The most significantly enriched terms are associated with translation and ribosome biogenesis, reflecting the enhanced protein synthesis machinery characteristic of cancer cells.

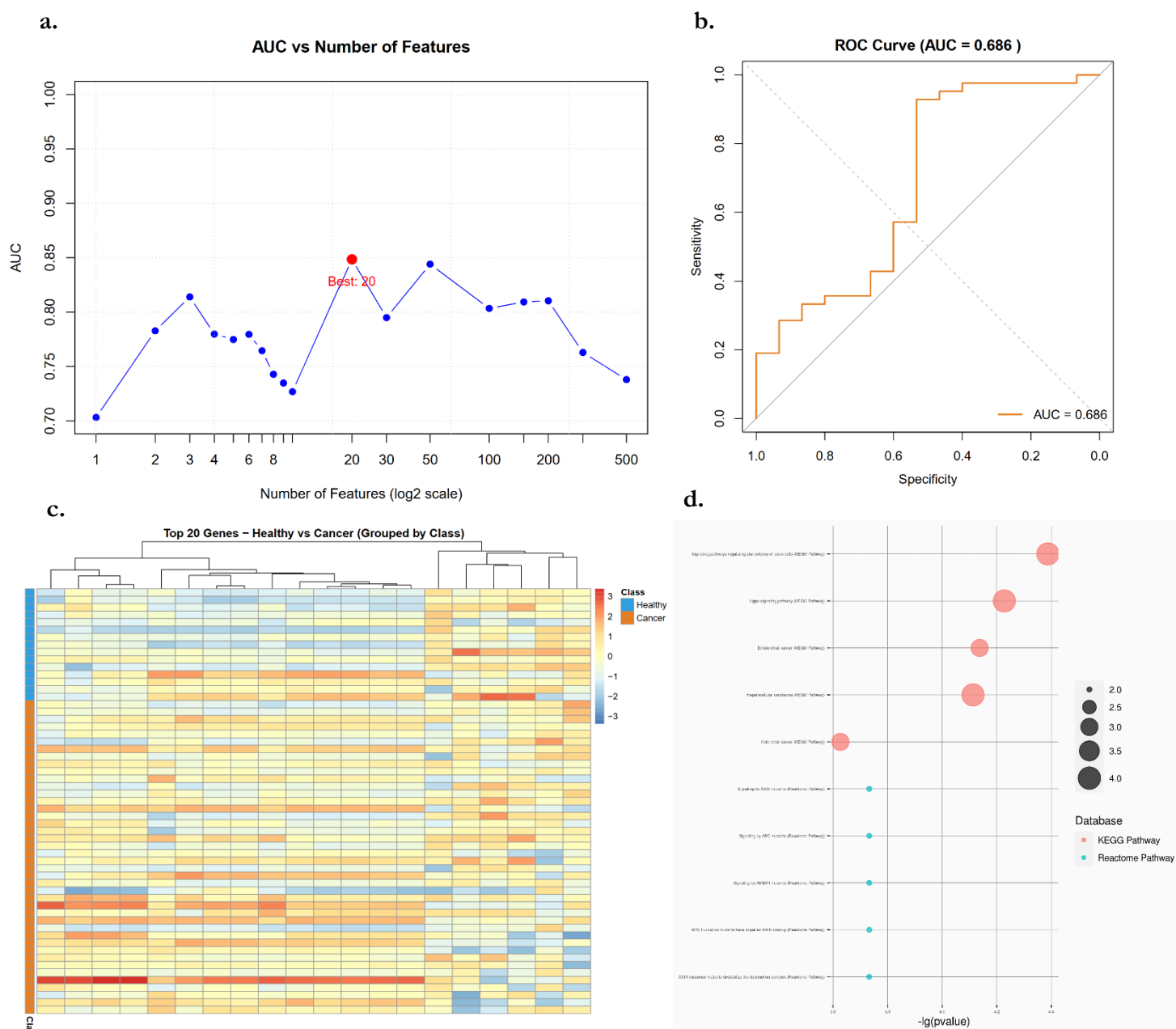


Figure 4. Machine learning results for the small RNA dataset. (a) Cross-validated AUC as a function of the number of top-ranked features included in the XGBoost model. The curve peaks at 20 features, which were selected as the optimal panel size. (b) ROC curve of the final XGBoost model evaluated on the held-out test set. The model achieved an AUC of 0.686, indicating moderate discriminative power between cancer and healthy samples. (c) Heatmap of the top 20 selected features on the test set samples, grouped by class (Healthy vs. Cancer). Samples are shown in rows and features in columns, with expression values scaled by column. Moderate separation between the two classes is observed, though the clustering is less pronounced than in the long RNA dataset. (d) Bubble plot of KEGG pathway enrichment results for the 7 miRNAs identified in the top 20 features, generated using RNAenrich. The size of each bubble represents the number of miRNAs annotated to the pathway, and the colour indicates the significance level. The most significantly enriched pathways include signaling pathways regulating pluripotency of stem cells and Hippo signaling pathway, both of which are closely linked to cancer development and progression.

5. Discussion

The primary objective of this study was to evaluate the feasibility of using exRNA expression profiles for cancer diagnosis and to identify a compact, biologically interpretable feature panel. I analyzed both long RNA and small RNA datasets derived from plasma samples, applying a consistent machine learning pipeline that integrated feature filtering, XGBoost classification, and functional interpretation.

The long RNA model achieved a test AUC of 0.891 with 200 features, substantially outperforming the small RNA model which reached an AUC of 0.686 with 20 features. Several factors may explain this performance gap. First, long RNA-seq provides a broader and more complex transcriptomic landscape, offering thousands of protein-coding and non-coding transcripts with potentially stronger differential expression signals between cancer and healthy states. In contrast, the small RNA repertoire is more limited in diversity and dynamic range, which may constrain the discriminative capacity of the model. Second, the long RNA dataset contained more samples (373 vs. 192), providing greater statistical power for model training and cross-validation. Third, the small RNA dataset required RUVg batch correction to remove technical variation from RNA isolation and library preparation steps; while effective, such corrections may also inadvertently remove some biological signal if it correlates with batch structure.

Despite its moderate predictive performance, the small RNA model yielded a functionally coherent feature set. The seven miRNAs identified in the top 20 features—

including miR-1246, miR-320 family members, and miR-193a—were enriched in cancer-relevant pathways such as signaling pathways regulating pluripotency of stem cells and the Hippo signaling pathway. This functional consistency reinforces the biological credibility of the selected features, even if the quantitative predictive power was moderate.

From a translational perspective, the long RNA panel appears more promising for further development as a diagnostic biomarker, given its superior AUC and clear separation of cancer from healthy samples in both PCA and heatmap visualizations. The small RNA panel, while less powerful individually, may still serve as a complementary layer in a multi-analyte liquid biopsy assay, especially given the stability and abundance of miRNAs in biofluids. The piRNA components identified in this dataset remain functionally uncharacterized due to the lack of comprehensive annotation tools for human piRNAs, highlighting a current limitation in the field.

In conclusion, this work demonstrates a complete bioinformatics workflow from raw data acquisition to feature identification and model evaluation. The long RNA signature, comprising 200 genes predominantly involved in translation and ribosome biogenesis, warrants further validation in independent cohorts. Future directions include integrating both long and small RNA features into a unified model, exploring additional classifiers such as random forest or support vector machines, and performing prospective validation using independent clinical samples.

Data Structure:

```
└─data
  │   └─CountMatrix
  │   │   └─long RNA-seq
  │   │       count.matrix.long.txt
  │   │       metadata.long.txt
  │   │
  │   └─small RNA-seq
  │       count.matrix.short.txt
  │       metadata.short.txt
  │
  └─Download
      Download.log
      thu_cloud_download.py
     
  └─result
      └─P2
          └─long
              logcpm.matrix.txt
              metadata.txt
              PCA.before.correction.pdf
              RLE.plot.pdf
              scaled.txt
          └─small
              boxplot.batch.effect.pdf
              logcpm.matrix.txt
              metadata.txt
              PCA.after.correction.pdf
              PCA.before.correction.pdf
              RLE.plot.pdf
              scaled.txt
          └─P3
              └─long
                  AUC.v.features.pdf
                  feature.lists.xlsx
                  filter.pvalues.all.txt
                  gost.results.txt
                  gost.top20.bubble.pdf
                  heatmap.top200.pdf
                  ROC.curve.pdf
              └─small
                  AUC.v.features.pdf
```

```
| | feature.lists.xlsx
| | filter.pvalues.all.txt
| | heatmap.top20.pdf
| | RNAenrich.bubble.plot.png
| | ROC.curve.pdf
| |
| └scripts
|   └long
|     | PartII.long.R
|     | PartIII.long.R
|     |
|     └small
|       PartII.small.R
|       PartIII.small.R
|
└report
  Report.docx
  Report.pdf
```

Codes:

[Python Script](#) (../data/Download/download.py):

```
# coding=utf-8

import os
import json
import requests
import sys

from urllib.parse import (
    urlparse,
    parse_qs,
    unquote,
    quote
)

class THUCloud:

    def __init__(self, shared_link, outdir="download"):

        self.headers = {
            "User-Agent":
            "Mozilla/5.0"
        }

        if "/d/" not in shared_link:
            raise ValueError("Only directory share links are supported.")

        archive = shared_link.split("/d/")[1].split("/")[0]

        self.api_link = (
            f"https://cloud.tsinghua.edu.cn/"
            f"api/v2.1/share-links/{archive}/dirents/"
        )

        self.file_link = (
            f"https://cloud.tsinghua.edu.cn/"
            f"d/{archive}/files/"
        )

        parsed = urlparse(shared_link)
        qs = parse_qs(parsed.query)
```

```

if "p" in qs:
    self.start_path = unquote(qs["p"][0])
else:
    self.start_path = "/"

self.root = os.path.abspath(outdir)

os.makedirs(self.root, exist_ok=True)

def list_dir(self, path):

    url = self.api_link + "?path=" + quote(path)

    r = requests.get(
        url,
        headers=self.headers,
        timeout=60
    )

    if r.status_code != 200:
        raise RuntimeError(
            f"Cannot access {path}\n"
            f"status={r.status_code}\n"
            f"{r.text}"
        )

    return r.json()["dirent_list"]

def download_file(self, remote_path, local_path):

    if os.path.exists(local_path):

        print("Skip:", local_path)

        return

    url = (
        self.file_link
        + "?p="
        + quote(remote_path)
        + "&dl=1"
    )

    r = requests.get(
        url,
        headers=self.headers,

```

```

        stream=True,
        timeout=60
    )

    if r.status_code != 200:
        print("Fail:", remote_path)
        return

    print("Downloading:", remote_path)

    with open(local_path, "wb") as f:

        for chunk in r.iter_content(
            chunk_size=1024 * 1024):

            if chunk:
                f.write(chunk)

    def recurse(self, remote_path, local_dir):

        os.makedirs(local_dir, exist_ok=True)

        items = self.list_dir(remote_path)

        for item in items:

            if item["is_dir"]:

                next_remote = item["folder_path"]

                next_local = os.path.join(
                    local_dir,
                    item["folder_name"]
                )

                self.recurse(
                    next_remote,
                    next_local
                )

            else:

                remote_file = item["file_path"]

                local_file = os.path.join(
                    local_dir,

```

```

        item["file_name"]
    )

    self.download_file(
        remote_file,
        local_file
    )

def run(self):

    print()

    print("Start path:")
    print(self.start_path)

    print()

    print("Output:")
    print(self.root)

    print()

    self.recurse(
        self.start_path,
        self.root
    )

    print()

    print("Finished.")

if __name__ == "__main__":

    if len(sys.argv) < 2:

        print()

        print(
            "Usage:\n"
            "python thu_cloud_download.py "
            "\"share_link\" "
            "[output_dir]"
        )

    sys.exit()

```



```
share = sys.argv[1]

if len(sys.argv) >= 3:
    out = sys.argv[2]
else:
    out = "./download"

THUCloud(
    share,
    out
).run()
```

Matrix Processing of Long RNAs (`../data/scripts/long/PartII.long.R`):

```
suppressPackageStartupMessages({
  library(edgeR)
  library(EDASeq)
  library(data.table)
})

# 1. Read matrix
count <- fread(
  "/root/exSEEK/CountMatrix/long RNA-seq/count.matrix.long.txt",
  data.table = FALSE
)

meta <- fread(
  "/root/exSEEK/CountMatrix/long RNA-seq/metadata.long.txt",
  data.table = FALSE
)

# 2. Adjust format
colnames(count)[-1] <- gsub("-PKU", "", colnames(count)[-1])
meta <- meta[match(colnames(count)[-1], meta$sample_id), ]

rownames(count) <- count[,1]
count <- count[,-1]

# 3. Filter genes with low expression
y <- DGEList(counts = count)
group <- factor(meta$label)

keep <- filterByExpr(y, group = group)
y <- y[keep, , keep.lib.sizes = FALSE]

# 4. Normalization
y <- edgeR::calcNormFactors(y, method = "TMM")
cpm.matrix <- edgeR::cpm(y)
logcpm.matrix <- edgeR::cpm(y, log = TRUE)

pdf(
  "/root/exSEEK/result/P2/long/RLE.plot.pdf",
  width = 12,
  height = 8
)
plotRLE(cpm.matrix)
dev.off()
```

```

# 5. Eliminate batch effect
expr <- t(logcpm.matrix)
pca <- prcomp(expr)

library_factor <- as.factor(meta$library)
library_numeric <- as.numeric(library_factor)
pdf("/root/exSEEK/result/P2/long/PCA.before.correction.pdf", width=10, height=8)
plot(pca$x[,1:2],
     col = library_numeric,
     pch = 19,
     main = "PCA Before Batch Correction",
     xlab = paste0("PC1 (", round(summary(pca)$importance[2,1]*100, 2), "%)"),
     ylab = paste0("PC2 (", round(summary(pca)$importance[2,2]*100, 2), "%)"))
legend("topright",
     legend = levels(library_factor),
     col = 1:length(levels(library_factor)),
     pch = 19)
dev.off()

# PCA plot before correction suggests that no correction is needed.

# 6. Data scaling
expr_scaled <- scale(expr, center = TRUE, scale = TRUE)

# 7. Save data
write.table(
  logcpm.matrix,
  "/root/exSEEK/result/P2/long/logcpm.matrix.txt",
  sep = "\t",
  quote = FALSE,
  row.names = TRUE
)

write.table(
  expr_scaled,
  "/root/exSEEK/result/P2/long/scaled.txt",
  sep = "\t",
  quote = FALSE,
  row.names = TRUE
)

write.table(
  meta,
  "/root/exSEEK/result/P2/long/metadata.txt",
  sep = "\t",
  quote = FALSE,

```

```
row.names = FALSE
```

```
)
```

Matrix Processing of Small RNAs (`../data/scripts/small/PartII.small.R`):

```
suppressPackageStartupMessages({  
  library(edgeR)  
  library(EDASeq)  
  library(data.table)  
  library(RUVSeq)  
  library(reshape2)  
  library(ggplot2)  
})  
  
# 1. Read matrix  
count <- fread(  
  "/root/exSEEK/CountMatrix/small RNA-seq/count.matrix.short.txt",  
  data.table = FALSE  
)  
  
meta <- fread(  
  "/root/exSEEK/CountMatrix/small RNA-seq/metadata.short.txt",  
  data.table = FALSE  
)  
  
# 2. Adjust format  
meta <- meta[match(colnames(count)[-1], meta[,1]), ]  
colnames(meta)[1] <- "sample_id"  
  
rownames(count) <- count[,1]  
count <- count[, -1]  
  
# 3. Filter genes with low expression  
y <- DGEList(counts = count)  
group <- factor(meta$label)  
  
keep <- filterByExpr(y, group = group)  
y <- y[keep, , keep.lib.sizes = FALSE]  
  
# 4. Normalization  
y <- edgeR::calcNormFactors(y, method = "TMM")  
cpm.matrix <- edgeR::cpm(y)  
logcpm.matrix <- edgeR::cpm(y, log = TRUE)  
  
pdf(  
  "/root/exSEEK/result/P2/small/RLE.plot.pdf",  
  width = 12,  
  height = 8  
)
```

```

plotRLE(cpm.matrix)
dev.off()

# 5. Check batch effect
expr <- t(logcpm.matrix)
pca <- prcomp(expr)

batch_cols <- c("RNA Isolation batch", "library preparation day", "gel cut size selection")
batch_labels <- c("RNA Isolation batch", "Library Preparation Day", "Gel Cut Size Selection")

pdf("/root/exSEEK/result/P2/small/PCA.before.correction.pdf", width = 15, height = 5)
par(mfrow = c(1, 3))

for (i in seq_along(batch_cols)) {
  batch_factor <- as.factor(meta[[batch_cols[i]]])
  batch_numeric <- as.numeric(batch_factor)

  plot(pca$x[,1:2],
       col = batch_numeric,
       pch = 19,
       main = batch_labels[i],
       xlab = paste0("PC1 (", round(summary(pca)$importance[2,1]*100, 2), "%)"),
       ylab = paste0("PC2 (", round(summary(pca)$importance[2,2]*100, 2), "%)"))

  legend("topright",
        legend = levels(batch_factor),
        col = 1:length(levels(batch_factor)),
        pch = 19,
        cex = 0.6)
}

dev.off()

# PCA plots of "Library Preparation Day" and "RNA Isolation Batch" before correction suggests that correction is
# needed.

# 6. Eliminate batch effect with RUVg
logcpm.matrix.before <- logcpm.matrix
logcpm_matrix_ruv <- as.matrix(logcpm.matrix)

cv.index <- apply(logcpm_matrix_ruv, 1, function(x) { sd(x) / mean(x) })
cv.index.sorted <- sort(cv.index)
empirical.control <- head(names(cv.index.sorted), as.integer(length(cv.index.sorted) * 0.2))

res <- RUVg(logcpm_matrix_ruv, empirical.control, k = 2, isLog = TRUE)
logcpm.matrix <- res$normalizedCounts

```

```

expr <- t(logcpm.matrix)

pca_corrected <- prcomp(expr)

pdf("/root/exSEEK/result/P2/small/PCA.after.correction.pdf", width = 15, height = 5)
par(mfrow = c(1, 3))

for (i in seq_along(batch_cols)) {
  batch_factor <- as.factor(meta[[batch_cols[i]]])
  batch_numeric <- as.numeric(batch_factor)

  plot(pca_corrected$x[,1:2],
       col = batch_numeric,
       pch = 19,
       main = paste("After RUVg -", batch_labels[i]),
       xlab = paste0("PC1 (", round(summary(pca_corrected)$importance[2,1]*100, 2), "%)"),
       ylab = paste0("PC2 (", round(summary(pca_corrected)$importance[2,2]*100, 2), "%)"))

  legend("topright",
        legend = levels(batch_factor),
        col = 1:length(levels(batch_factor)),
        pch = 19,
        cex = 0.6)
}
dev.off()

sum_logcpm_before <- colSums(logcpm.matrix.before)
sum_logcpm_after <- colSums(logcpm.matrix)

pdf("/root/exSEEK/result/P2/small/boxplot.batch.effect.pdf", width = 12, height = 12)
par(mfrow = c(2, 2))

# RNA Isolation Batch
boxplot(sum_logcpm_before ~ meta$`RNA Isolation batch`,
       main = "RNA Isolation Batch",
       xlab = "Batch",
       ylab = "Sum of logCPM",
       col = "#2E9FDF",
       las = 1)
boxplot(sum_logcpm_after ~ meta$`RNA Isolation batch`,
       main = "RNA Isolation Batch After RUVg",
       xlab = "Batch",
       ylab = "Sum of logCPM",
       col = "#E67E22",
       las = 1)

```

```

# Library Preparation Day
boxplot(sum_logcpm_before ~ meta$`library preparation day`,
        main = "Library Preparation Day",
        xlab = "Batch",
        ylab = "Sum of logCPM",
        col = "#2E9FDF",
        las = 1)

boxplot(sum_logcpm_after ~ meta$`library preparation day`,
        main = "Library Preparation Day After RUVg",
        xlab = "Batch",
        ylab = "Sum of logCPM",
        col = "#E67E22",
        las = 1)

dev.off()

# 7. Data scaling
expr_scaled <- scale(expr, center = TRUE, scale = TRUE)

# 8. Save data
write.table(
  logcpm.matrix,
  "/root/exSEEK/result/P2/small/logcpm.matrix.txt",
  sep = "\t",
  quote = FALSE,
  row.names = TRUE
)

write.table(
  expr_scaled,
  "/root/exSEEK/result/P2/small/scaled.txt",
  sep = "\t",
  quote = FALSE,
  row.names = TRUE
)

write.table(
  meta,
  "/root/exSEEK/result/P2/small/metadata.txt",
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)

```

Machine Learning of Long RNAs ([../data/scripts/long/PartIII.long.R](#)):

```
suppressPackageStartupMessages({
  library(data.table)
  library(caret)
  library(openxlsx)
  library(xgboost)
  library(pROC)
  library(pheatmap)
  library(gprofiler2)
  library(ggplot2)
})

# 1. Read matrix
expr_scaled <- fread(
  "/root/exSEEK/result/P2/long.scaled.txt",
  data.table = FALSE
)
rownames(expr_scaled) <- expr_scaled[,1]
expr_scaled <- expr_scaled[,-1]

logcpm_matrix <- fread(
  "/root/exSEEK/result/P2/long.logcpm.matrix.txt",
  data.table = FALSE
)
rownames(logcpm_matrix) <- logcpm_matrix[,1]
logcpm_matrix <- logcpm_matrix[,-1]

meta <- fread(
  "/root/exSEEK/result/P2/long.metadata.txt",
  data.table = FALSE
)
meta$class <- ifelse(meta$label == "NC", "Healthy", "Cancer")
meta$class <- factor(meta$class, levels = c("Healthy", "Cancer"))

# 2. Divide between train/test sets
set.seed(42)
train_index <- createDataPartition(
  y = meta$class,
  p = 0.7,
  list = FALSE
)

X_train <- expr_scaled[train_index, ]
X_test <- expr_scaled[-train_index, ]
```

```

y_train <- meta$class[train_index]
y_test <- meta$class[-train_index]

meta_train <- meta[train_index, ]
meta_test <- meta[-train_index, ]

# 3. Feature selection via filter
p_values <- numeric(ncol(X_train))
names(p_values) <- colnames(X_train)

for (i in 1:ncol(X_train)) {
  cancer_expr <- X_train[y_train == "Cancer", i]
  healthy_expr <- X_train[y_train == "Healthy", i]
  test_result <- t.test(cancer_expr, healthy_expr, alternative = "two.sided")
  p_values[i] <- test_result$p.value
}

p_values_sorted <- sort(p_values)
sig_genes <- names(p_values[p_values < 0.05])

top_n_list <- list(
  top_50 = names(p_values_sorted[1:50]),
  top_100 = names(p_values_sorted[1:100]),
  top_200 = names(p_values_sorted[1:200]),
  top_500 = names(p_values_sorted[1:500]),
  top_1000 = names(p_values_sorted[1:1000])
)

pvalue_df <- data.frame(
  gene = names(p_values_sorted),
  p_value = p_values_sorted,
  rank = 1:length(p_values_sorted)
)

write.table(
  pvalue_df,
  "/root/exSEEK/result/P3/long/filter.pvalues.all.txt",
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)

feature_counts <- c(1:10, 20, 30, 50, 100, 200)
feature_lists <- list()

for (n in feature_counts) {
  feature_lists[[paste0("top", n)]] <- data.frame(
    rank = 1:n,

```

```

        gene = names(p_values_sorted[1:n]),
        p_value = p_values_sorted[1:n]
    )
}

write.xlsx(feature_lists, "/root/exSEEK/result/P3/long/feature.lists.xlsx")

# 4. AUC v feature count
y_train_num <- ifelse(y_train == "Cancer", 1, 0)
y_test_num <- ifelse(y_test == "Cancer", 1, 0)

params <- list(
  objective = "binary:logistic",
  eval_metric = "auc",
  max_depth = 6,
  eta = 0.1,
  subsample = 0.8,
  colsample_bytree = 0.8,
  min_child_weight = 1,
  scale_pos_weight = sum(y_train_num == 0) / sum(y_train_num == 1)
)

feature_counts <- c(1:10, 20, 30, 50, 100, 200, 500, 1000)
auc_curve <- numeric(length(feature_counts))

for (i in seq_along(feature_counts)) {
  n <- feature_counts[i]
  genes <- names(p_values_sorted[1:n])

  X_train_sub <- as.matrix(X_train[, genes])
  X_test_sub <- as.matrix(X_test[, genes])

  dtrain_sub <- xgb.DMatrix(data = X_train_sub, label = y_train_num)
  dtest_sub <- xgb.DMatrix(data = X_test_sub, label = y_test_num)

  set.seed(42)
  cv_sub <- xgb.cv(
    params = params,
    data = dtrain_sub,
    nrounds = 200,
    nfold = 5,
    early_stopping_rounds = 20,
    verbose = 0,
    prediction = FALSE
  )

  best_idx <- which.max(cv_sub$evaluation_log$test_auc_mean)

```

```

    auc_curve[i] <- max(cv_sub$evaluation_log$test_auc_mean)
  }

pdf("/root/exSEEK/result/P3/long/AUC.v.features.pdf", width = 8, height = 6)
log_features <- log2(feature_counts)

plot(log_features, auc_curve, type = "b", pch = 19, col = "blue",
     xlab = "Number of Features (log2 scale)", ylab = "AUC",
     main = "AUC vs Number of Features",
     ylim = c(0.7, 1.0), xaxt = "n")

axis(1, at = log_features, labels = feature_counts)
grid()

best_idx <- which.max(auc_curve)
points(log_features[best_idx], auc_curve[best_idx], col = "red", pch = 19, cex = 1.5)
text(log_features[best_idx], auc_curve[best_idx] - 0.02,
     labels = paste0("Best: ", feature_counts[best_idx]), col = "red", cex = 0.9)

dev.off()

# 5. Model training: XGBoost
top200_genes <- names(p_values_sorted[1:200])

X_train_top <- X_train[, top200_genes]
X_test_top <- X_test[, top200_genes]

X_train_matrix <- as.matrix(X_train_top)
X_test_matrix <- as.matrix(X_test_top)

dtrain <- xgb.DMatrix(data = X_train_matrix, label = y_train_num)
dtest <- xgb.DMatrix(data = X_test_matrix, label = y_test_num)

set.seed(42)
cv_result <- xgb.cv(
  params = params,
  data = dtrain,
  nrounds = 200,
  nfold = 5,
  early_stopping_rounds = 20,
  verbose = 1,
  prediction = FALSE
)

eval_log <- cv_result$evaluation_log
best_auc <- max(eval_log$test_auc_mean)

```

```

best_nrounds <- which.max(eval_log$test_auc_mean)

set.seed(42)
xgb_model <- xgb.train(
  params = params,
  data = dtrain,
  nrounds = best_nrounds,
  verbose = 1
)

pred_prob <- predict(xgb_model, dtest)
pred_class <- ifelse(pred_prob > 0.5, "Cancer", "Healthy")
pred_class <- factor(pred_class, levels = c("Healthy", "Cancer"))

cm_xgb <- confusionMatrix(pred_class, y_test)

roc_xgb <- roc(y_test, pred_prob)
auc_xgb <- auc(roc_xgb)

pdf("/root/exSEEK/result/P3/long/ROC.curve.pdf", width = 6, height = 6)
plot(roc_xgb, main = paste("ROC Curve (AUC =", round(auc_xgb, 3), ")"),
     col = "#E67E22", lwd = 2)
abline(a = 0, b = 1, lty = 2, col = "gray")
legend("bottomright", legend = paste("AUC =", round(auc_xgb, 3)),
     col = "#E67E22", lwd = 2, bty = "n")
dev.off()

# 6. Heatmap of top features
expr_heat <- logcpm_matrix[top200_genes, rownames(X_test)]
expr_heat <- t(expr_heat)

annotation_df <- data.frame(
  Class = y_test
)
rownames(annotation_df) <- rownames(expr_heat)

healthy_idx <- which(y_test == "Healthy")
cancer_idx <- which(y_test == "Cancer")
sample_order <- c(healthy_idx, cancer_idx)

expr_heat <- expr_heat[sample_order, ]
annotation_df <- annotation_df[sample_order, , drop = FALSE]

ann_colors <- list(
  Class = c(Healthy = "#2E9FDF", Cancer = "#E67E22")
)

```

```

pdf("/root/exSEEK/result/P3/long/heatmap.top200.pdf", width = 10, height = 8)
pheatmap(
  expr_heat,
  scale = "column",
  clustering_distance_rows = "euclidean",
  clustering_distance_cols = "euclidean",
  clustering_method = "complete",
  annotation_row = annotation_df,
  annotation_colors = ann_colors,
  show_rownames = FALSE,
  show_colnames = FALSE,
  main = "Top 200 Genes - Healthy vs Cancer (Grouped by Class)",
  fontsize = 10,
  cluster_rows = FALSE,
  cluster_cols = TRUE
)
dev.off()

# 7. Function analysis
top200_genes_clean <- sub("\\\\.*", "", top200_genes)

gost_results <- gost(
  query = top200_genes_clean,
  organism = "hsapiens",
  sources = c("GO", "KEGG", "REAC"),
  user_threshold = 0.05,
  correction_method = "g_SCS",
  evcodes = TRUE
)
results_df <- gost_results$result

results_df_clean <- results_df
results_df_clean$evidence_codes <- sapply(results_df_clean$evidence_codes, paste, collapse = ";")
results_df_clean$parents <- sapply(results_df_clean$parents, paste, collapse = ";")

write.table(
  results_df_clean,
  "/root/exSEEK/result/P3/long/gost.results.txt",
  sep = "\\t",
  quote = FALSE,
  row.names = FALSE
)

top20 <- results_df[order(results_df$p_value), ][1:20, ]
top20$term_name <- factor(top20$term_name, levels = rev(top20$term_name))

```

```
pdf("/root/exSEEK/result/P3/long/gost.top20.bubble.pdf", width = 10, height = 7)
ggplot(top20, aes(x = -log10(p_value), y = term_name)) +
  geom_point(aes(size = intersection_size, color = source)) +
  scale_color_manual(values = c(
    "GO:BP" = "#8E44AD",
    "GO:CC" = "#27AE60",
    "REAC" = "#2E9FDF"
  )) +
  theme_minimal() +
  labs(
    title = "Top 20 Enriched GO Terms",
    x = "-log10(p-value)",
    y = "",
    size = "Gene Count",
    color = "Source"
  ) +
  theme(legend.position = "right")
dev.off()
```

```
suppressPackageStartupMessages({
  library(data.table)
  library(caret)
  library(openxlsx)
  library(xgboost)
  library(pROC)
  library(pheatmap)
  library(gprofiler2)
  library(ggplot2)
})

# 1. Read matrix
expr_scaled <- fread(
  "/root/exSEEK/result/P2/small/scaled.txt",
  data.table = FALSE
)
rownames(expr_scaled) <- expr_scaled[,1]
expr_scaled <- expr_scaled[,-1]

logcpm_matrix <- fread(
  "/root/exSEEK/result/P2/small/logcpm.matrix.txt",
  data.table = FALSE
)
rownames(logcpm_matrix) <- logcpm_matrix[,1]
logcpm_matrix <- logcpm_matrix[,-1]

meta <- fread(
  "/root/exSEEK/result/P2/small/metadata.txt",
  data.table = FALSE
)
meta$class <- ifelse(meta$label == "HD", "Healthy", "Cancer")
meta$class <- factor(meta$class, levels = c("Healthy", "Cancer"))

# 2. Divide between train/test sets
set.seed(42)
train_index <- createDataPartition(
  y = meta$class,
  p = 0.7,
  list = FALSE
)

X_train <- expr_scaled[train_index, ]
X_test <- expr_scaled[-train_index, ]
```



```

y_train <- meta$class[train_index]
y_test <- meta$class[-train_index]

meta_train <- meta[train_index, ]
meta_test <- meta[-train_index, ]

# 3. Feature selection via filter
p_values <- numeric(ncol(X_train))
names(p_values) <- colnames(X_train)

for (i in 1:ncol(X_train)) {
  cancer_expr <- X_train[y_train == "Cancer", i]
  healthy_expr <- X_train[y_train == "Healthy", i]
  test_result <- t.test(cancer_expr, healthy_expr, alternative = "two.sided")
  p_values[i] <- test_result$p.value
}

p_values_sorted <- sort(p_values)
sig_genes <- names(p_values[p_values < 0.05])

top_n_list <- list(
  top_50 = names(p_values_sorted[1:50]),
  top_100 = names(p_values_sorted[1:100]),
  top_200 = names(p_values_sorted[1:200]),
  top_500 = names(p_values_sorted[1:500]),
  top_1000 = names(p_values_sorted[1:1000])
)

pvalue_df <- data.frame(
  gene = names(p_values_sorted),
  p_value = p_values_sorted,
  rank = 1:length(p_values_sorted)
)

write.table(
  pvalue_df,
  "/root/exSEEK/result/P3/small/filter.pvalues.all.txt",
  sep = "\t",
  quote = FALSE,
  row.names = FALSE
)

feature_counts <- c(1:10, 20, 30, 50, 100, 150, 200, 300, 500)
feature_lists <- list()

for (n in feature_counts) {
  feature_lists[[paste0("top", n)]] <- data.frame(
    rank = 1:n,

```

```

        gene = names(p_values_sorted[1:n]),
        p_value = p_values_sorted[1:n]
    )
}

write.xlsx(feature_lists, "/root/exSEEK/result/P3/small/feature.lists.xlsx")

# 4. AUC v feature count
y_train_num <- ifelse(y_train == "Cancer", 1, 0)
y_test_num <- ifelse(y_test == "Cancer", 1, 0)

params <- list(
  objective = "binary:logistic",
  eval_metric = "auc",
  max_depth = 6,
  eta = 0.1,
  subsample = 0.8,
  colsample_bytree = 0.8,
  min_child_weight = 1,
  scale_pos_weight = sum(y_train_num == 0) / sum(y_train_num == 1)
)

feature_counts <- c(1:10, 20, 30, 50, 100, 150, 200, 300, 500)
auc_curve <- numeric(length(feature_counts))

for (i in seq_along(feature_counts)) {
  n <- feature_counts[i]
  genes <- names(p_values_sorted[1:n])

  X_train_sub <- as.matrix(X_train[, genes])
  X_test_sub <- as.matrix(X_test[, genes])

  dtrain_sub <- xgb.DMatrix(data = X_train_sub, label = y_train_num)
  dtest_sub <- xgb.DMatrix(data = X_test_sub, label = y_test_num)

  set.seed(42)
  cv_sub <- xgb.cv(
    params = params,
    data = dtrain_sub,
    nrounds = 200,
    nfold = 5,
    early_stopping_rounds = 20,
    verbose = 0,
    prediction = FALSE
  )

  best_idx <- which.max(cv_sub$evaluation_log$test_auc_mean)

```

```

    auc_curve[i] <- max(cv_sub$evaluation_log$test_auc_mean)
  }

pdf("/root/exSEEK/result/P3/small/AUC.v.features.pdf", width = 8, height = 6)
log_features <- log2(feature_counts)

plot(log_features, auc_curve, type = "b", pch = 19, col = "blue",
     xlab = "Number of Features (log2 scale)", ylab = "AUC",
     main = "AUC vs Number of Features",
     ylim = c(0.7, 1.0), xaxt = "n")

axis(1, at = log_features, labels = feature_counts)
grid()

best_idx <- which.max(auc_curve)
points(log_features[best_idx], auc_curve[best_idx], col = "red", pch = 19, cex = 1.5)
text(log_features[best_idx], auc_curve[best_idx] - 0.02,
     labels = paste0("Best: ", feature_counts[best_idx]), col = "red", cex = 0.9)

dev.off()

# 5. Model training: XGBoost
top20_genes <- names(p_values_sorted[1:20])

X_train_top <- X_train[, top20_genes]
X_test_top <- X_test[, top20_genes]

X_train_matrix <- as.matrix(X_train_top)
X_test_matrix <- as.matrix(X_test_top)

dtrain <- xgb.DMatrix(data = X_train_matrix, label = y_train_num)
dtest <- xgb.DMatrix(data = X_test_matrix, label = y_test_num)

set.seed(42)
cv_result <- xgb.cv(
  params = params,
  data = dtrain,
  nrounds = 200,
  nfold = 5,
  early_stopping_rounds = 20,
  verbose = 1,
  prediction = FALSE
)

eval_log <- cv_result$evaluation_log
best_auc <- max(eval_log$test_auc_mean)

```

```

best_nrounds <- which.max(eval_log$test_auc_mean)

set.seed(42)
xgb_model <- xgb.train(
  params = params,
  data = dtrain,
  nrounds = best_nrounds,
  verbose = 1
)

pred_prob <- predict(xgb_model, dtest)
pred_class <- ifelse(pred_prob > 0.5, "Cancer", "Healthy")
pred_class <- factor(pred_class, levels = c("Healthy", "Cancer"))

cm_xgb <- confusionMatrix(pred_class, y_test)

roc_xgb <- roc(y_test, pred_prob)
auc_xgb <- auc(roc_xgb)

pdf("/root/exSEEK/result/P3/small/ROC.curve.pdf", width = 6, height = 6)
plot(roc_xgb, main = paste("ROC Curve (AUC =", round(auc_xgb, 3), ")"),
     col = "#E67E22", lwd = 2)
abline(a = 0, b = 1, lty = 2, col = "gray")
legend("bottomright", legend = paste("AUC =", round(auc_xgb, 3)),
     col = "#E67E22", lwd = 2, bty = "n")
dev.off()

# 6. Heatmap of top features
expr_heat <- logcpm_matrix[top20_genes, rownames(X_test)]
expr_heat <- t(expr_heat)

annotation_df <- data.frame(
  Class = y_test
)
rownames(annotation_df) <- rownames(expr_heat)

healthy_idx <- which(y_test == "Healthy")
cancer_idx <- which(y_test == "Cancer")
sample_order <- c(healthy_idx, cancer_idx)

expr_heat <- expr_heat[sample_order, ]
annotation_df <- annotation_df[sample_order, , drop = FALSE]

ann_colors <- list(
  Class = c(Healthy = "#2E9FDF", Cancer = "#E67E22")
)

```

```
pdf("/root/exSEEK/result/P3/small/heatmap.top20.pdf", width = 10, height = 8)
pheatmap(
  expr_heat,
  scale = "column",
  clustering_distance_rows = "euclidean",
  clustering_distance_cols = "euclidean",
  clustering_method = "complete",
  annotation_row = annotation_df,
  annotation_colors = ann_colors,
  show_rownames = FALSE,
  show_colnames = FALSE,
  main = "Top 20 Genes - Healthy vs Cancer (Grouped by Class)",
  fontsize = 10,
  cluster_rows = FALSE,
  cluster_cols = TRUE
)
dev.off()
```